

API Security Risk Analysis Report:

ReqRes Products Endpoint

Tool Used: Insomnia REST Client

Framework: OWASP API Security Top 10

How an API Security Breach is Initiated

An API security breach occurs when attackers exploit weaknesses in how an application communicates with its database or other services. Attackers use tools like Insomnia to send specially crafted requests to endpoints, looking for ways to bypass "locked doors" like authentication or project-level restrictions. If successful, an attacker can gain access to sensitive internal data, tenant information, or infrastructure details that should never be public.

Common API Risk Indicators found in ReqRes

- **Broken Authorization (BOLA):** The API sometimes ignores specific identifiers (like UUIDs), returning all records instead of just one, which allows a user to "see everything".
- **Parameter Manipulation:** Attackers can change a `project_id` in the URL to a random number. If the API doesn't verify this against your login key, they can view data belonging to other projects.
- **Information Disclosure:** Sometimes the "behind-the-scenes" data—like internal timestamps, server types, or trial status—is leaked in the response headers or body.
- **Infrastructure Exposure:** Response headers may reveal exactly where the app is hosted (e.g., Heroku or Cloudflare), giving attackers a roadmap of which systems to target.

API Risk Classification

High Risk (Immediate Action Required)

An API finding is considered High Risk when it represents a critical flaw in access control. For example, failing to enforce `project_id` scoping (F-002) allows any user to potentially view another tenant's private data. This is a cross-tenant leakage risk that must be fixed immediately.

Medium Risk (Short-term Priority)

A finding is Medium Risk if it bypasses intended security boundaries or exposes infrastructure.

- F-001: UUID path parameters are ignored, allowing full collection enumeration.
- F-004: Infrastructure stack (Heroku/Cloudflare) is disclosed in headers.
- F-006: A CORS wildcard (*) is used, allowing any website to make requests to the API.

Low Risk (Long-term Improvement)

A finding is Low Risk when it leaks non-critical internal metadata or trial information. While not an immediate path to a breach, fields like `created_by` or internal A/B testing variants (F-007) provide attackers with "fingerprinting" data to understand the system's internal workflows.

Impact of API Security Weaknesses

- Unauthorized Data Access: Attackers can enumerate all records or access other projects.
- System Mapping: Leaked headers allow attackers to identify known vulnerabilities in the specific infrastructure stack.
- Resource Exhaustion: Improperly configured rate limits (F-008) can allow attackers to pace their attacks without being blocked.

Prevention and Remediation Methods

- Enforce Scoping: Always validate that the user's API key has permission to access the specific `project_id` requested.
- Use DTOs: Implement a serialization layer to strip out internal fields like `collection_id` or `deleted_at` before the data reaches the user.
- Restrict CORS: Replace the wildcard (*) with a specific allowlist of trusted domains.
- Mask Headers: Use tools like Cloudflare to hide "server" and "via" headers that reveal your backend technology.

Conclusion

While the ReqRes API has strong foundations—like enforcing API keys and using HSTS—the current authorization bypasses (BOLA) represent a meaningful threat. By implementing server-side filtering and stripping internal metadata, the platform can significantly reduce its attack surface and protect its multi-tenant environment.